CS 419/519 — Special Topics: Image Processing and Computer Vision

Geometric Operations

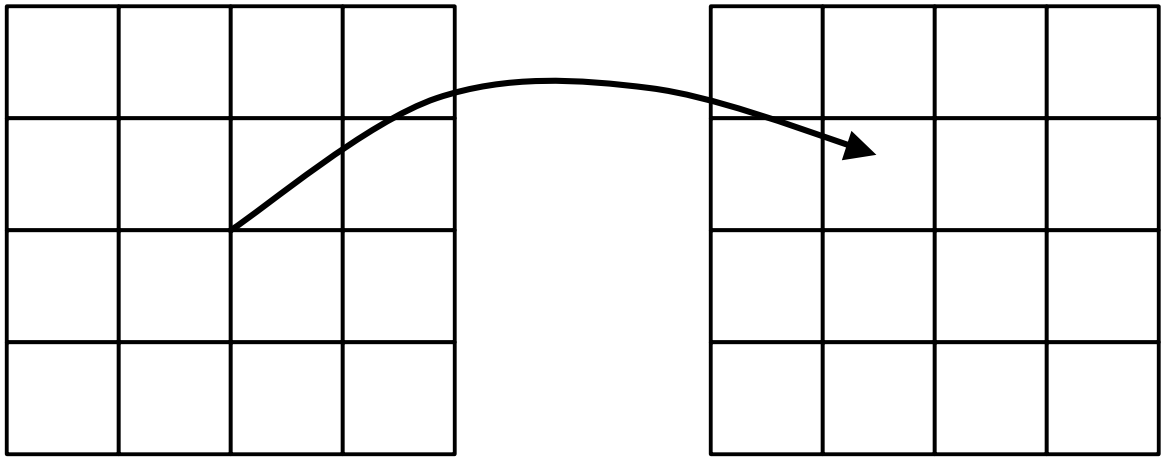
Geometric Operations

- Transformations: Shift, Rotation, etc.
- Resizing (Scaling).
- Adding/Correcting a Warp.
- Texture Mapping
- Morphing

Forward Mapping

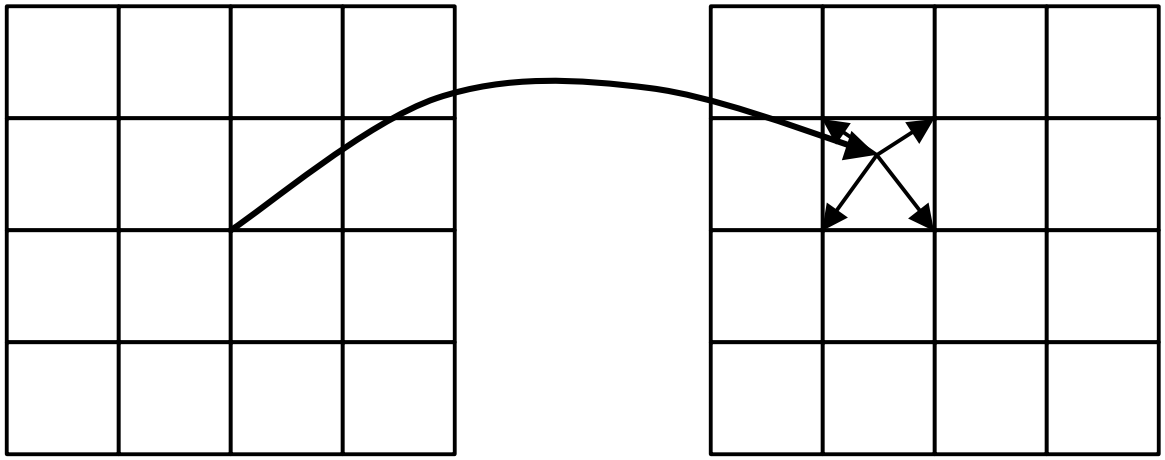
Let $u(x, y)$ and $v(x, y)$ be a mapping from location (x, y) to (u, v):

$$B[u(x, y), v(x, y)] = A[x, y]$$



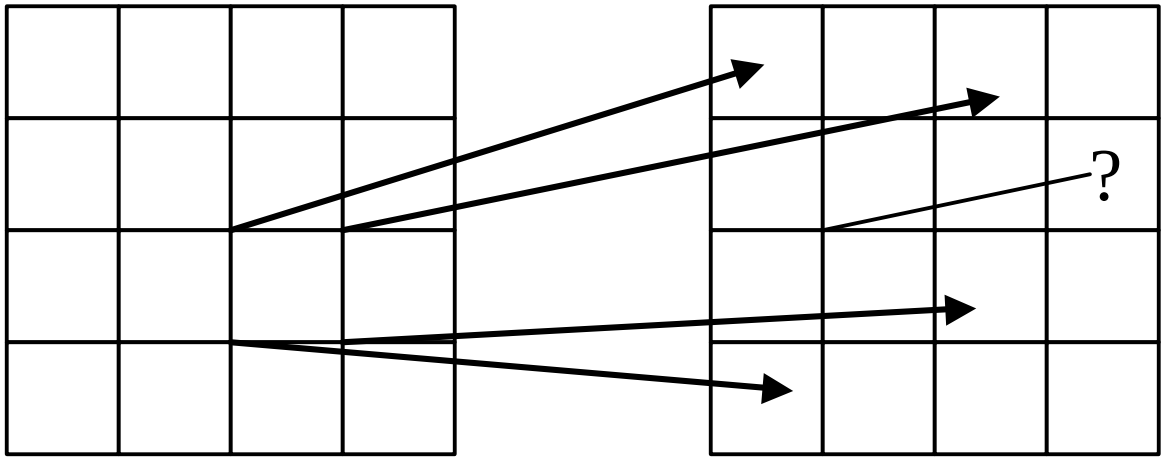
Forward Mapping: Problems

- Doesn’t always map *to* pixel locations.
- Solution: spread out effect of each pixel.



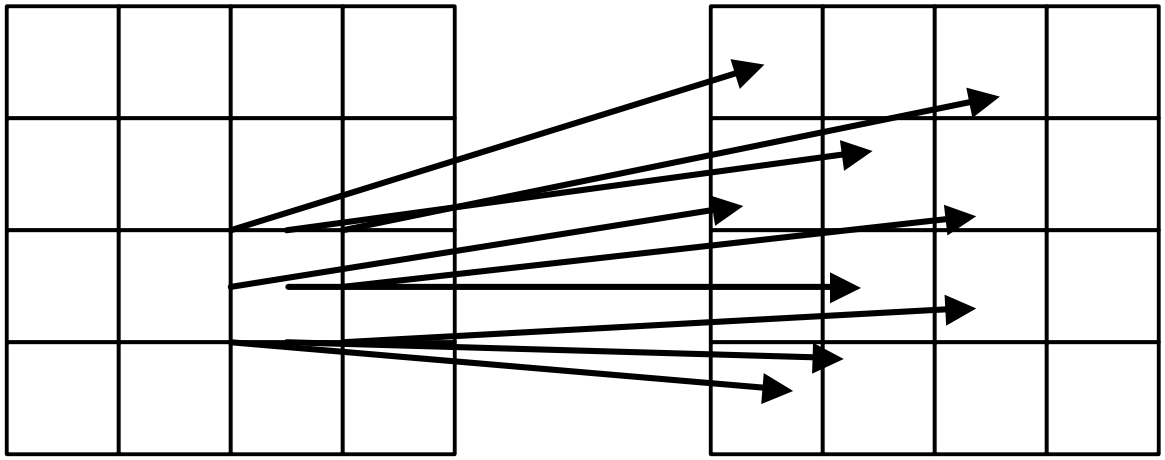
Forward Mapping: Problems

- Map produce holes in the output.



Forward Mapping: Problems

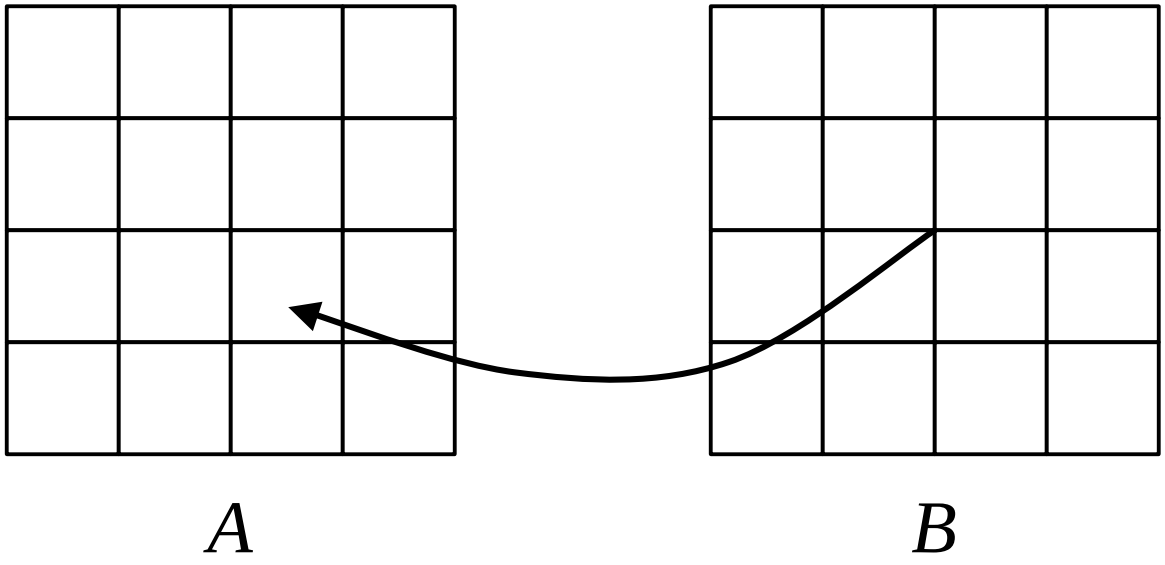
- May produce holes in the output.
- Solution: sample source image (*A*) more often.
  - Still can leave holes.



Backward Mapping

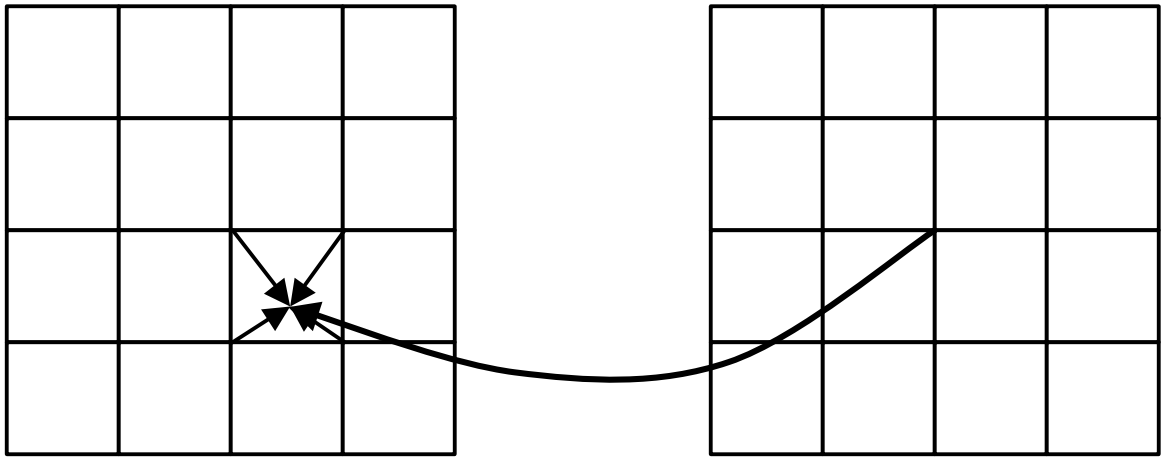
Let $x(u, v)$ and $y(u, v)$ be an inverse mapping from location (x, y) to (u, v):

$$B[u, v] = A[x(u, v), y(u, v)]$$



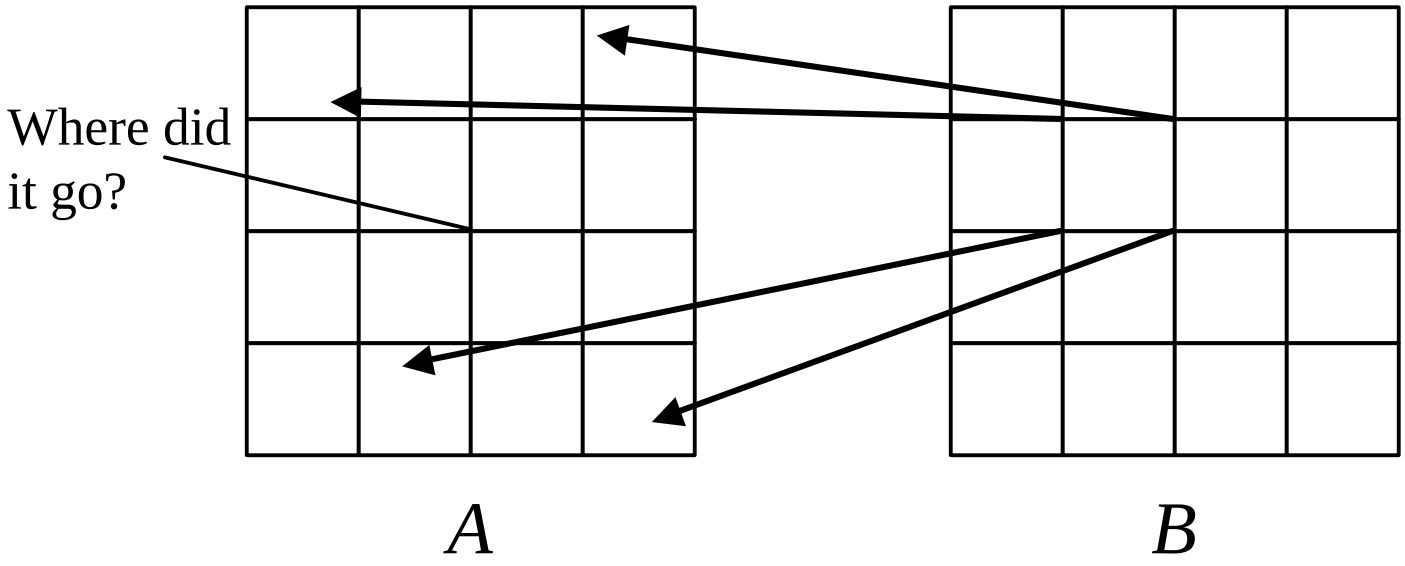
Backward Mapping: Problems

- Doesn’t always map *from* a pixel.
- Solution: Interpolate between pixels.



Backward Mapping: Problems

- May produce holes in the input.
- Solution: reduce input image (by averaging pixels) and sample reduced/averaged image → MIP-maps

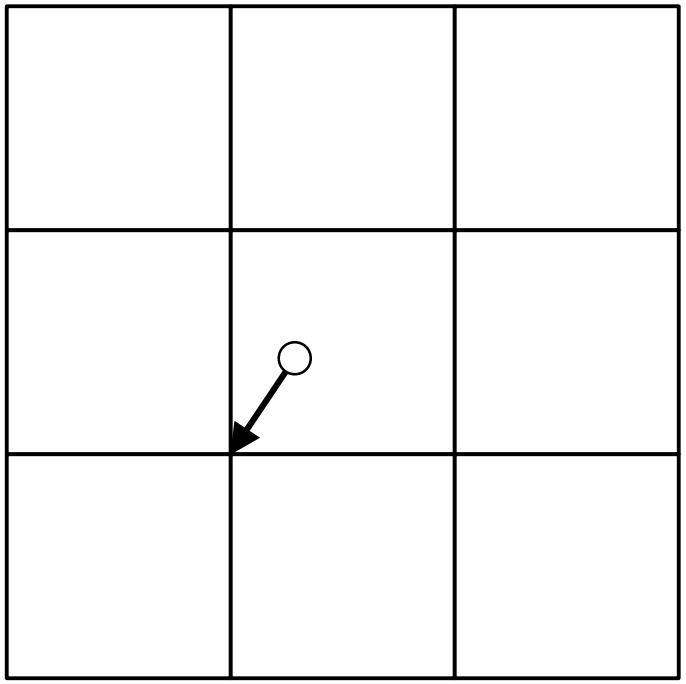


Interpolation

- “Filling In” between the pixels
- A function of the neighbors or a larger neighborhood.
- Methods:
  - Nearest neighbor
  - Bilinear
  - Bicubic or other higher-order

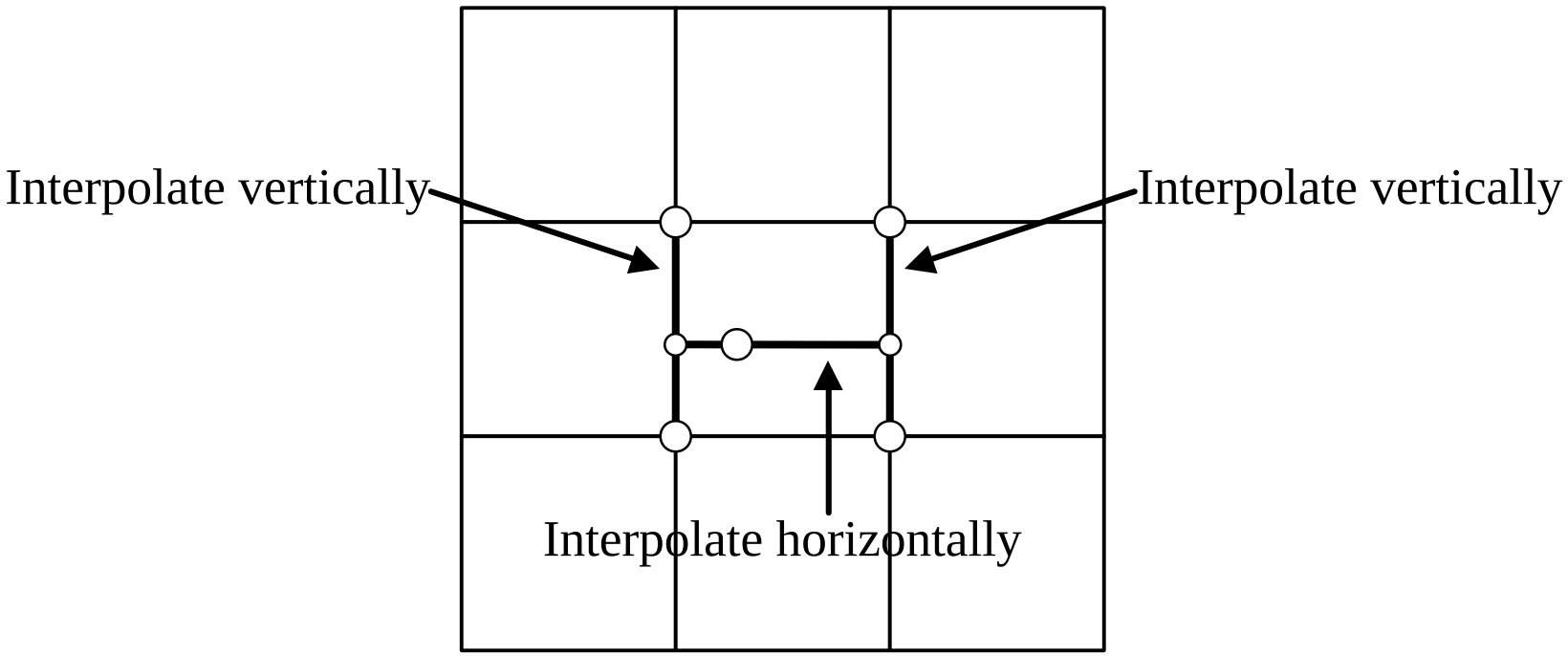
Nearest-Neighbor Interpolation

- Simplest to implement
- Round off *x* and *y* values to nearest pixel.
- Result is not continuous (blocky).



Bilinear Interpolation

- Linearly interpolate in one direction (e.g., vertically)
- Linearly interpolate results in the other direction (horizontally)



Higher-Order Interpolation

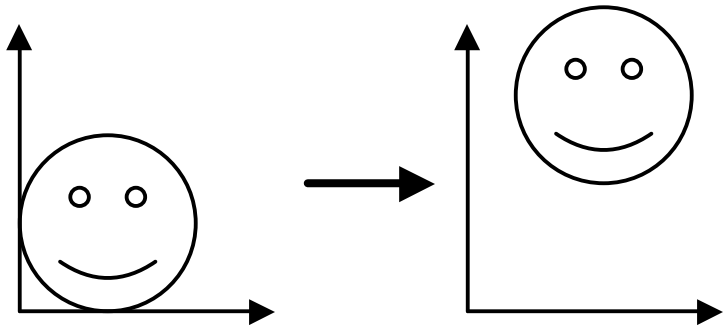
- Higher-degree polynomials:
  - e.g., bicubic (cubic interpolation vertically followed by cubic interpolation horizontally).
- Sometimes other interpolating functions.
- Requires a larger neighborhood:
  - e.g., bicubic requires a 4×4 neighborhood.
- More computational

Geometric Operations: Translation

Shifting left-right and/or up-down:

$$x' = x + x_0$$

$$y' = y + y_0$$



Matrix form:

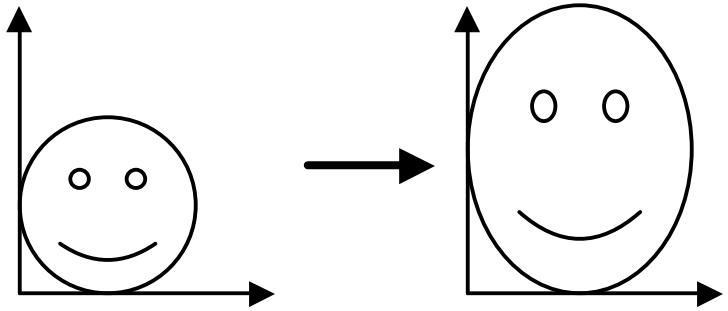
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + x_0 \\ y + y_0 \\ 1 \end{bmatrix}$$

Geometric Operations: Scaling

Enlarging or reducing horizontally and/or vertically:

$$x' = S_x x$$

$$y' = S_y y$$



Matrix form:

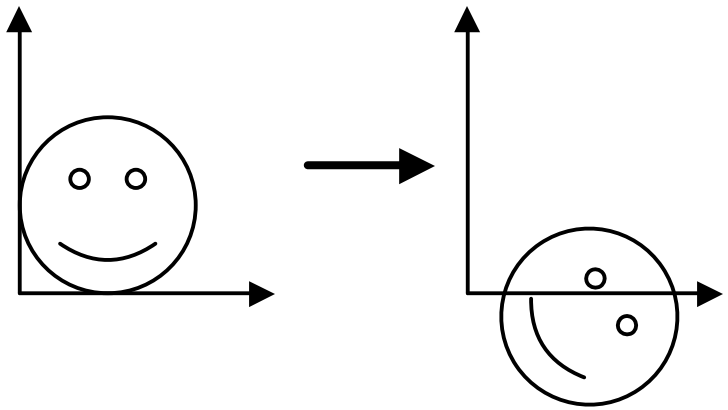
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} S_x x \\ S_y y \\ 1 \end{bmatrix}$$

Geometric Operations: Rotation

Result components dependent on *both* x & y:

$$x' = \cos(q)\,x - \sin(q)\,y$$

$$y' = \sin(q)\,x + \cos(q)\,y$$



Matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(q) & -\sin(q) & 0 \\ \sin(q) & \cos(q) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(q)\,x - \sin(q)\,y \\ \sin(q)\,x + \cos(q)\,y \\ 1 \end{bmatrix}$$

Geometric Operations: Affine Transform

Linear combinations of x, y, and 1: encompasses translation, scaling, & rotation.

$$x¢ = a\,x + by + c$$

$$y¢ = d\,y + ey + f$$

Matrix form:

$$\begin{bmatrix} x¢ \\ y¢ \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a\,x + by + c \\ d\,y + ey + f \\ 1 \end{bmatrix}$$

Affine Transformations (cont.)

- Translation, rotation, and scale are *rigid* body transformations:
  - Their transformation matrices are orthogonal.
- General affine transformations also include non-rigid transformations (e.g., skew or shear).
  - Affine means that parallel lines transform to parallel lines.

Multiple Transformations

Example: rotation around the point (x_0, y_0).

$$\begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(q) & -\sin(q) & 0 \\ \sin(q) & \cos(q) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Matrix multiplication is associative (but not commutative):

$$\boldsymbol{B}(\boldsymbol{A}\boldsymbol{v}) = (\boldsymbol{B}\boldsymbol{A})\boldsymbol{v} = \boldsymbol{C}\boldsymbol{v}$$

where $C = BA$.

- Can compose multiple transformations into a single matrix.
- Much faster when applying same transform to many pixels.

Inverting Matrix Transformations

If

$$v¢ = M\,v$$

then

$$v = M^{-1}v¢$$

Thus, to invert the transformation, invert the matrix.

Useful for computing the backward mapping given the forward transform.

Warp Meshes

- For more elaborate or free-form warps, we can create a mesh of warped points.
- Input mesh points transform to corresponding output mesh points.
- Interpolate in between mesh points to get the transformation.

Morphing

- Instead of a warping mesh, use arbitrary correspondence points:
  - Tip of one person's nose to the tip of another, eyes to eyes, etc.
- Interpolate between correspondence points to determine where how points move.
- Apply standard warping (forward or backward):
  - In-between image is a weighted average of the source and destination corresponding pixels.